Prayog — Concurrency Test Report

CS455 Deliverable 3 · Group 3 · 6 November 2026

1. Summary

Result: in every experiment and test, no GPU unit was ever allocated twice. The control in section 6 double-booked in 99 of 100 rounds without the protection.

Prayog's correctness under concurrency rests on PostgreSQL row locks (`FOR UPDATE`, `FOR UPDATE SKIP LOCKED`, `FOR SHARE`) and partial unique indexes, not on application-level locks: several API workers, the scheduler tick and node agents act on the same rows at once. A new suite, `test_cases/gpu/test_concurrency.py`, attacks the six places where a race would do damage. Every test releases its threads at the same instant through a `threading.Barrier`, and every thread uses its own database session, as separate API requests would.

Test	Race	Invariant	Result
C1	the same `client_request_id` submitted by 16 threads at once	exactly one job; every caller gets the same request ID (FR-GPU-107)	**pass**
C2	40 parallel submissions by one user with `max_active_jobs_per_user = 5`	exactly 5 jobs created; the other 35 refused with a rule-named error, none with an unexpected exception	**pass**
C3	8 scheduling passes at once over 10 queued jobs (1–2 GPUs) and a 4-unit node	no unit in two live allocations; no job in two; every `SCHEDULED` job owns the units it holds; each unit's `current_allocation_id` matches its live allocation	**pass**
C4	25 wrong passwords for one account at once (limit 10)	exactly 10 "failed credentials" events, exactly 1 lockout, 15 "locked out"; the right password then refused	**pass**
C5	an owner's cancel racing 4 scheduling passes for the same 2-GPU job	either CANCELLED and holding nothing, or placed and with a stop requested — never CANCELLED while holding units	**pass**
C6	a pass meeting a unit whose row disagrees with its live allocation	the pass does not crash; the job is placed on the healthy node	**failed on `main` → defect D3-03 → fixed → pass**

Each test was run three times in a row on a dedicated database (`prayog_d3_conc`) and once more for the recorded junit file: **6/6 passed in all four runs** (`evidence/junit_concurrency.xml`). In the final full-suite run on a shared database, C3's last assertion failed: the passes had also placed its jobs on *other tests*' shared nodes, which the assertion did not expect. The assertion was corrected to count the jobs' live allocations on any node (no product change), and C3–C6 then passed on both the shared and the dedicated database. C6 was also run against the unfixed `claim.py`, where it fails — so the test can detect the defect it guards against.

2. How the tests are built

```
def _together(n, fn):
    gate = threading.Barrier(n)            # all n threads start their call at the same moment
    def call(i):
        gate.wait()
        try:
            return fn(i)
        except Exception as exc:           # collected, asserted on by the test
            return exc
    with ThreadPoolExecutor(n) as pool:
        return list(pool.map(call, range(n)))
```

- **Real database, real locks.** The suite needs the migrated PostgreSQL test database (`PRAYOG_RUN_SCHEDULER_POSTGRES_TESTS=1`), as in CI; it is skipped without it.

- **No ORM object crosses a thread.** IDs are read before the threads start. The first version of the suite read `user.id` inside the threads, which made SQLAlchemy refresh an expired object through the shared test session and fail with "*This session is provisioning a new connection; concurrent operations are not permitted*" — a bug in the test, not the product (AI Engineering Log section 4.1).
- **Isolation from other tests.** Rows carry a per-test suffix. Because a scheduling pass considers *every* queued job in the database, assertions are about the test's own jobs and units only, and C6 uses nodes reserved to the test's lab.

3. What each lock protects (and the test that shows it)

Mechanism	Code	Shown by
Per-user serialisation of submissions, so the active-job limit and the idempotency check cannot both be passed by two requests	`submit_job` locks the user's scheduling context (`logic/gpu/scheduler/requests.py`)	C1, C2
Job row `FOR UPDATE SKIP LOCKED`: a job being claimed by another pass is skipped, not waited for	`claim.py` step 1	C3, C5
Node row `FOR SHARE`: claims run side by side but wait for a state change (drain, down)	`claim.py` step 2	C3
Unit rows `FOR UPDATE SKIP LOCKED`, lowest index first; fewer than *k* locked = `LOST_CLAIM_RACE`	`claim.py` step 3	C3
Partial unique index `ux_alloc_units_live`: one live allocation per unit, the backstop if a path skips the locks	migration `2c898af77bf1_gpu_scheduling_tables`	C6 (the backstop fires)
User row `FOR UPDATE` across password verification, so failed-login counts are exact	`logic/auth.py:124`	C4
Cancel and release through `lifecycle.request_stop` / `release` on locked rows	`logic/gpu/scheduler/cancel.py`, `lifecycle.py`	C5

4. Defect found: D3-03 — one inconsistent unit stops all scheduling

Found by: running C3 on the shared test database after the full backend suite. A row left by another test had a live `allocation_units` row for unit 289 while the unit's own `current_allocation_id` was NULL. Every pass then chose unit 289 (it *looks* free), the insert hit `ux_alloc_units_live`, and the `IntegrityError` escaped `run_pass`:

```
IntegrityError: duplicate key value violates unique constraint "ux_alloc_units_live"
DETAIL:  Key (unit_id)=(289) already exists.
```

Because every pass meets the same unit first, **one inconsistent row stops scheduling for the whole pool**, not just for that node. The index did its job (no double allocation), but the pass treated the backstop firing as fatal.

How it could happen in production: any future code path that writes allocations without going through `claim`, a manual database repair, or a partial restore. The index exists precisely because that is possible (claim.py docstring).

Fix (`logic/gpu/scheduler/claim.py`): an `IntegrityError` from the claim's transaction is rolled back, logged at ERROR with the job and node, and returned as `LOST_CLAIM_RACE`; the pass tries the job's next node and continues with the other jobs. The inconsistent unit stays unusable (and visible in the log) until an admin repairs it.

Regression test: C6 — a "bad" node with the disagreement and a healthy node, both reserved to the test's lab; two passes must not raise and the waiting job must end on the healthy node.

5. Concurrency covered elsewhere

Race	Where
Double delivery of outbox events (ledger consumed once)	`test_mlflow_outbox`, `test_mlflow_ledger_consumer` (unique `event_key`)
Single-use diagnostic links consumed twice	`consume_diagnostic_link` locks the link row; `test_diagnostic_session_audit` (replay)
Steward single-use updates	Steward suite (`UPDATE … RETURNING` guard, Steward log error #3)
Agent report racing the reaper or an admin action	reaper docstring: every action through `lifecycle` on locked rows; failure drills F1, F4
Parallel API load (64 users)	Performance report — exposed pool exhaustion (D3-07), not a data race

6. The reproducible experiment: many users, one resource

The tests above release threads inside one process. `backend/experiments/concurrency_experiment.py` repeats the challenge end to end, against the deployed stack (four API worker processes, the tick running), and shows the race is real by removing the protection.

Part A: through the API. Real tokens, real HTTP, four worker processes.

Scenario	Rounds	Rounds wrong	Double allocations	Jobs scheduled per round	GPUs held per round	Slowest fill
40 users ask for 1 GPU each; 4 GPUs exist	10	0	0	[4]	[4]	0.63 s
10 users ask for 3 GPUs each; 4 exist (all or nothing)	10	0	0	[1]	[3]	0.141 s

Part B: claims racing for the last unit.

Claimers	Rounds	Rounds with exactly one winner	Outcomes	Claim p95
4 (NFR-GPU-038: at most 100 ms)	20	20	{'LOST_CLAIM_RACE': 60, 'CLAIMED': 20}	17.95 ms
16	20	19	{'LOST_CLAIM_RACE': 300, 'CLAIMED': 19, 'JOB_UNAVAILABLE': 1}	44.07 ms

Four passes at once at the design peak (40 jobs, 20 units), 10 rounds: 0 rounds broke an invariant, pass p95 333.73 ms.

Part C: why this strategy. Eight workers race for one unit, on small tables of their own, with no artificial delay.

Strategy	Rounds double-booked	Attempts refused by the database	Attempt p95 (ms)
check then write, no lock, no index	99 of 100	0	25.39
check then write, partial unique index only	0 of 100	697	12.08
SERIALIZABLE transactions, no lock	0 of 100	698	12.4
FOR UPDATE (waits for the lock)	0 of 100	0	23.65
FOR UPDATE SKIP LOCKED plus index (Prayog)	0 of 100	0	7.84

- **Check then write, with no protection,** double-books nearly every time. This is the race the course asks us to explain: both transactions see the unit free before either writes.
- **A unique index alone** is correct, but every loser gets an error, and a pass then has to treat errors as normal flow. We keep it as the backstop.
- **SERIALIZABLE** is also correct, but it aborts the losers and needs retry logic around every claim.

- `FOR UPDATE` is correct, but the workers queue behind each other, even though a loser can do nothing useful once it gets the lock.
- `FOR UPDATE SKIP LOCKED` **plus the index,** the strategy Prayog uses, is correct, has no errors, and never waits: a claimer that cannot get a unit moves on at once.

Property test (PRAYOG-105). `test_cases/gpu/scheduler/test_allocation_properties.py` uses Hypothesis to apply random sequences of submit, pass, start, finish, node failure and cancel on a real database, and checks every allocation invariant after every step. `test_scoring.py` checks both starvation bounds the same way.

7. Not covered (yet)

- Two *processes* running the scheduler tick at once (the deployment runs exactly one; a second would be safe by the locks above, but it is not tested).
- Concurrency between the node agent's local state file and its own restarts beyond the existing "nothing is run twice" test.